AI ENGINEER / AGENTS & AUTOMATION

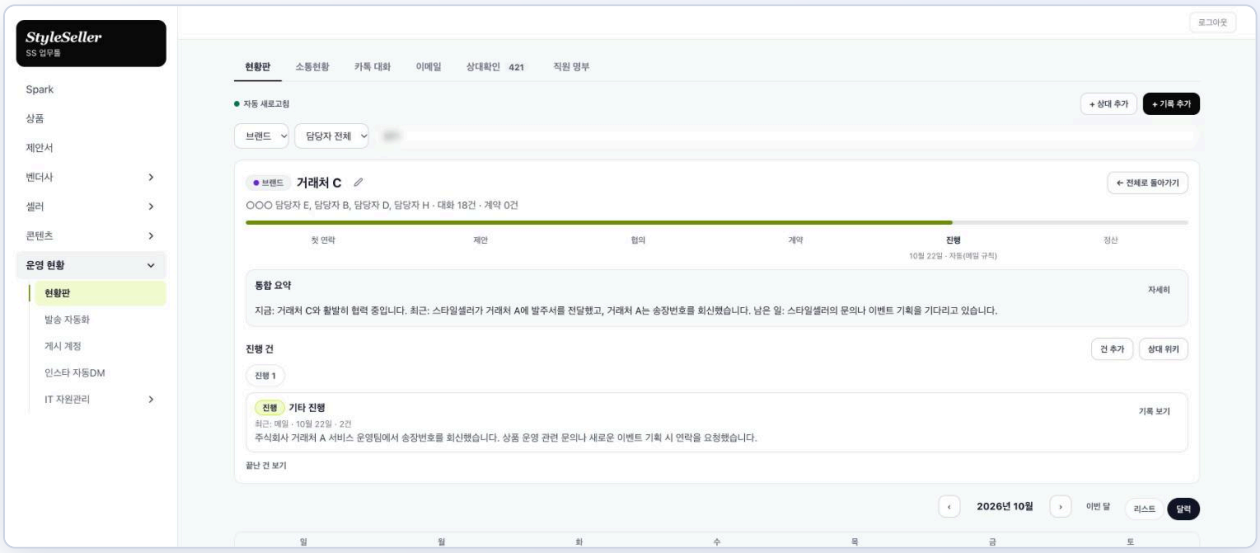
Portfolio

“Turning repeated work into AI systems that run every day”

Jinhee Mok

AI Engineer · from design to deployment and operations

I turn repetitive office work into AI systems that run every day. I build and run them. StyleSeller is a commerce platform that operates sales for creator sellers and brands.



Status Board in production, Screen A: the per-counterpart calendar

01
Status Board
352 counterparts on the board
KakaoTalk · email · Slack →
Masking · LLM extraction → Staff confirms

02
Vendor CS Agent
107 test files
Staff-PC collection →
Knowledge hub search → Mac send · delivery check

03
Internal Work Tool
43 price-calculation test cases
Catalog · partner pool → LLM candidates → Staff confirms

04
Knowledge Hub · Email Agent
13 knowledge workspaces
Sent mail · chat wiki · PDFs →
LLM planner → Drafts · answers

05
Agent Platform · Spark
109 shared skills
Request → Hermes dispatcher → Publisher

06
Content Autopilot
~300 test files
Request → Render · QA →
Publish gate

07
Staff PC Fleet
11 staff PCs
Staff Mac · Windows → VM →
Watch cron · Slack

Jinhee Mok
jinheemok815@gmail.com

https://jin-tonix.github.io
https://github.com/Jin-tonix



Portfolio

jinheemok815@gmail.com
https://github.com/Jin-tonix

Contact

Jinhee Mok
jinheemok815@gmail.com
https://jin-tonix.github.io
https://github.com/Jin-tonix

Skills

• Used in production (StyleSeller, 2026-02 onward)

- AI / Agent
  Tool-calling agents, MCP servers I built, GraphRAG (LightRAG · Neo4j · pgvector), LiteLLM proxy (Gemini path), Execution gate (Tool Registry: dry run, approval row recheck), Multiple Claude Code sessions
- Backend
  Python, FastAPI, TypeScript, Next.js 16, Node.js / Express, PostgreSQL / PostgREST, Supabase (moved to self-hosting in 2026-08)
- Frontend
  React 19, JavaScript, HTML, CSS, Tailwind 4
- Staff PCs · infra
  Chrome Extension MV3, Playwright / CDP, Swift (macOS), PowerShell, launchd, Docker, Docker Compose, Docker Swarm, GitHub Actions, GCP VM, nginx, Zero-downtime swap deploy script (for the brand-tool move to the VM), n8n, Git, GitHub

• Used in training programs (2024)

- Full-stack course
  Java, Spring Boot, MySQL, JPA, Vue.js, Flutter
- AI course
  LangChain RAG chatbot, OCR and translation pipeline

About me

"I'm Jinhee Mok, an AI engineer who turns repetitive work into AI systems."

I take the work people repeat by hand every day, move it into AI agents and automation, and then ship and run those systems myself.

I joined StyleSeller in November 2025 and work as a PL in the business team. Starting from seller DMs, I built and now run vendor support, the work tool, the email agent and knowledge hub, Spark · Work OS, and staff PC deployment.

Claude Code writes a lot of my code, but what to build and whether it works are my calls. Nothing is done until it has run on a real staff PC.

I studied law, taught English, and managed a daycare before moving into software; a We:Review internship automating receipts with OCR + LLM started this work.

I want to build workplaces where systems take the repetition and people keep the judgment.

Journey

From a law degree to AI Engineer (Agents & Automation)

2025-11 – present	**StyleSeller · PL (business team)** Design, build, and run the internal AI work systems
2025-08 – 2025-10	**We:Review (Welead) · intern** Built an OCR + LLM receipt review pipeline
2025-05 – 2025-07	**Korea IT Business Promotion Association** Generative AI developer training program
2024-10 – 2024-12	**HiMedia Academy · AI course** AI projects with industry mentors (LangChain RAG chatbot, OCR and translation pipeline)
2024-04 – 2024-10	**HiMedia Academy · full-stack course** Career Change Spring Boot and Vue.js team projects; collaboration and version control
2021-02 – 2023-06	**Academic Credit Bank System · Business Administration** Business administration degree program
2018-08 – 2024-02	**Monte Kids · Btop Kinder · Jungsang Academy · English teacher** Lesson design, teaching materials, and parent consultations
2015-03 – 2018-02	**Jeonwon Daycare Center · operations manager** Staff management and day-to-day operations
2008-03 – 2012-02	**Cheongju University · B.A. in Law** Law major

At a glance

Seven systems I designed, built, and run at StyleSeller. The pages that follow show, case by case, how each one works and why I built it that way. Each figure carries the date it was counted.

Status Board · Operations Ledger

Pick one counterpart and its KakaoTalk, email, and order sheets land on one calendar; the deal stage of all 352 counterparts fits on one screen

Vendor CS Agent

A KakaoTalk chatbot that answers only tagged questions, never hands off, and rewrites any number or brand that does not match the evidence

Internal Work Tool · Pricing, Matching, Personalized KakaoTalk

Code computes the amounts; the LLM only returns candidates and the fields to change

LightRAG Knowledge Hub · Email Agent

Four agents pull evidence from one knowledge hub, and only evidence-backed email drafts reach staff drafts folders

Agent Platform · Spark

No matter who asks, work starts as a work order from the CEO agent, and the server controls delegation and tool execution

Content Autopilot

From a Reels request to a 9:16 MP4 render and approval, all in the operator's screen. Live SNS posting is simulated

Staff PC Deployment · Operations

KakaoTalk and Instagram work has no official API, so I ship it to 11 staff PCs with a one-line install and run it there (as registered on 2026-09-16)

Status Board

Status Board · Operations Ledger

StyleSeller · private company repositories

Pick one counterpart and its KakaoTalk, email, and order sheets land on one calendar; the deal stage of all 352 counterparts fits on one screen

Period	Built 2026.10.19~10.23 (5 days), in production
Role	Design: ledger schema (a table that stores each record exactly once), counterpart matching rules / Build: intake API, email collector, 6 UI tabs / Verification: DB tests, production screens. I used AI coding tools; I set the scope and checked the results with tests
Repositories	Status Board commits: about 100 in brand-tool + about 50 in agent-company

Background

Conversations with counterparts are split across KakaoTalk rooms, the order@ mailbox, and Slack, and Excel order sheets arrive separately as email attachments. The Status Board summarizes these records per day and places them in date order on a per-counterpart calendar. Pick one brand, vendor, or seller, and every record exchanged with that counterpart comes together on one screen. The Communication Status tab shows where each of the 352 counterparts stands, from first contact to settlement. Records the rules could not match, and stages that are unclear, are decided by staff.

Structure and scale

1. Tech stack

Next.js 16.2·React 19.2·Tailwind 4·Vitest 4 · Python 3.12·Pydantic v2 · Postgres RLS (DB rules that limit who can see each row) · Gemini Flash family (via LiteLLM)

2. Scale

- Communication Status: first contact 46, proposal 119, negotiation 9, contract 16, in progress 162, settlement 0 (2026-10-24)
- 352 counterparts = 147 brands + 135 vendors + 70 sellers (2026-10-24)
- Roughly 100–200 records per weekday, 421 awaiting counterpart matching (2026-10-24)

3. How it works

① KakaoTalk comes in 3 times a day (09:15·13:00·17:55), order@ email every 5 minutes, and Slack through a generic connector. However many times the same record arrives, the ledger keeps it once.

② Newsletters are filtered out by rules before any LLM call. In order-sheet tables, columns such as recipient are located and buyer names are masked. If a phrase the summary cites as evidence is not in the original text, the summary is discarded.

③ Counterpart matching uses rules only, no LLM. It checks email thread, business registration number, KakaoTalk room, email, phone, domain, account code, and alias, in that order. If keys disagree, the record goes to Unclassified.

④ In-progress deals are built by grouping KakaoTalk contract signals and email signals under a counterpart·target·round key. A stage is applied automatically when the rule evidence is clear; the rest go to 'awaiting review' for staff to decide.

⑤ Every day at 18:05, summaries for up to 300 counterparts are rewritten in 3–6 lines. If spend passes ₩2,000 for the day or the cost ledger cannot be read, that day's batch stops.

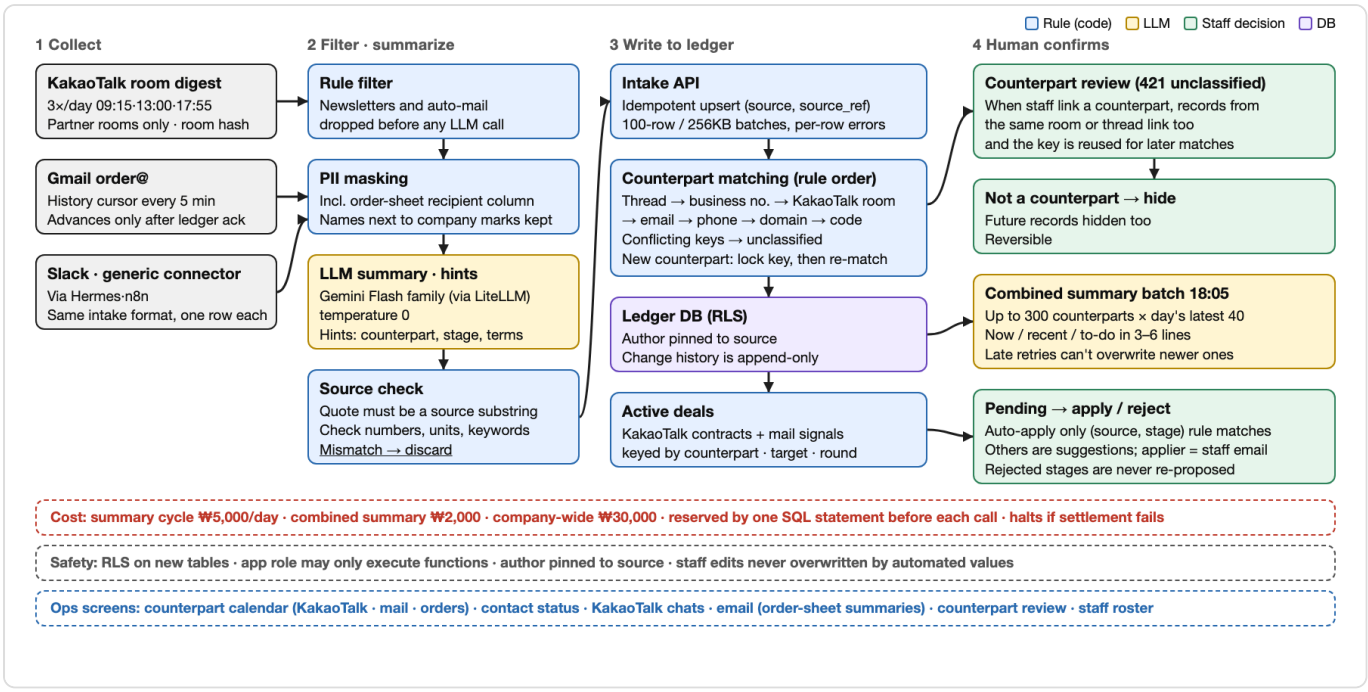
Incidents and fixes

1. **All 107 summaries rejected (2026-10-23).** The app role had no execute permission on the DB function that stores summaries. All 107 summaries that day failed at the save step with db_error. A one-line GRANT giving execute permission on that one function fixed it.

2. **Too many 'in progress' proposals in the rehearsal (2026-10-23).** The cause was a rule that read any mention of words like 'shipping·inventory·sales' as a deal signal. I kept only real deal-signal words, and changed matching to strip negative phrases such as '판매중단' (sales halted) and '발주문의' (order inquiry) first.

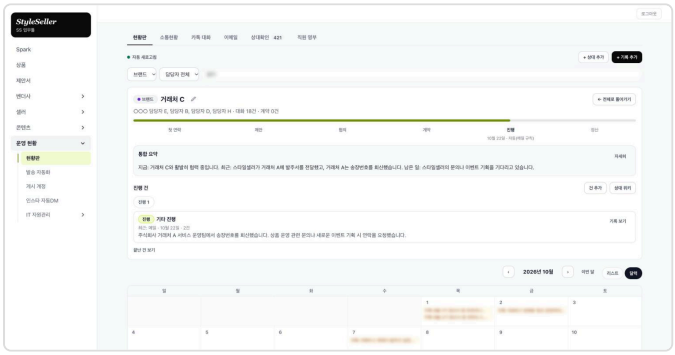
3. **Shared chatbot shown as the contact person (2026-10-23).** The shared chatbot's name ended up in the contact-person field for 64 counterparts. The top 5 most active speakers in a KakaoTalk room were taken as contacts, and the chatbot was among them. Now only company email accounts (@styleseller.co.kr) count as contacts; if there is none, the contact from the account profile is used.

Architecture

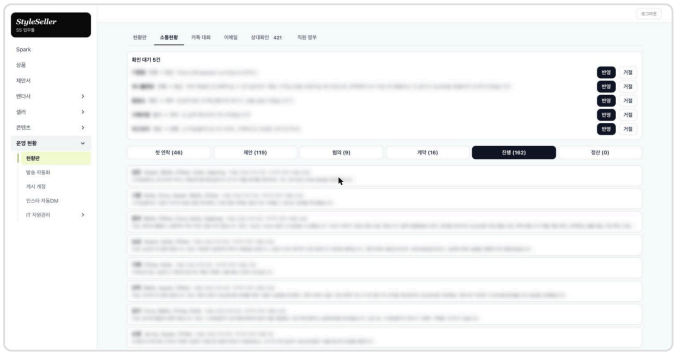


KakaoTalk·email·Slack → agent-company (masking, LLM extraction, check against original text) → idempotent intake (stored once no matter how many times it arrives) → brand-tool ledger (RLS, rule matching) → staff review

Screens



Screen A. Per-counterpart calendar. Pick one brand, vendor, or seller, and KakaoTalk summaries, emails, and Excel order sheets sit on a single calendar in date order. Above it are a stage bar, a combined summary (now/recent/open items), in-progress deals, and the counterpart wiki.



Screen B. Communication Status. Shows counterpart counts per stage (first contact 46 to in progress 162) and 'awaiting review' proposals; staff click apply or reject.

Highlights

1. **The screen stays the same when the same record arrives again.** Collectors resend the same record any number of times. It is stored with a single upsert statement that updates when (source, source_ref) matches. Rows edited by staff, and rows whose content is unchanged, are left alone. When content is unchanged the trigger does not bump the sequence number or timestamp either, so the screen stays the same after a resend.

2. **The author value is never taken from the caller.** The intake function drops the author value the caller sends and rebuilds it as 'ingest:source'. The manual source is staff-only, so it is rejected. The 'edited by staff' flag is set by a trigger, not the app, and hidden records cannot be restored with UPDATE. The app role gets only execute permission on functions.

3. **When keys disagree, nothing is matched.** The matching order is fixed: thread, business registration number, KakaoTalk room, email, phone, domain, account code, alias. If a business registration number or phone points to a different counterpart than the other evidence, it is flagged as a conflict and left in Unclassified. 13 public domains such as gmail and naver are not used as evidence.

4. **Concurrent collection does not create a counterpart twice.** Only when creating a new counterpart, an advisory lock (a named lock shared across transactions) is taken per evidence value in sorted order, then the lookup runs again. If another transaction created the counterpart in the meantime, that one is used. Staff stage updates follow the same lock order as collection (counterpart → proposal), so there are no deadlocks.

5. **LLM output is checked against the original text.** A stage evidence phrase is kept only if it is a substring of the masked body. Deal terms are stricter. Every number in the value must appear in the quote, and the keywords and units for that term type must match too. A company name is used only if its normalized name appears in the evidence line.

6. **Order-sheet tables are masked by column.** Order sheets usually arrive as tables. Header columns such as 'recipient·orderer' are located, and values in those columns are masked only on rows with the same cell count as the header row. Product names and quantities therefore stay in the summary. A name next to a corporate marker such as '㈜' is treated as a company name and not masked.

7. **Cost is reserved before the call.** The balance check and the reservation are one SQL statement, so two concurrent calls cannot both pass the limit. The reserved amount is the larger of an estimate based on maximum input/output and a floor (the minimum reservation). If settlement fails, later calls stop. A late retried summary fails the date comparison and cannot overwrite a newer summary.

8. **The mail cursor moves only after the ledger has received everything.** Emails go into a local queue first and are then sent to the ledger. The Gmail history cursor moves only when every delivery in that round succeeds. If the cursor expires, the gap is backfilled by listing from the last successful time. An email that fails twice is quarantined and raises an alert.

Status, judgment, and limits

- In production. The 6 tabs, KakaoTalk·email·Slack intake, and the 18:05 summary batch are running (checked on the production screens on 2026-10-24).

- Python, TS, and SQL share one golden file for name and phone normalization. CI runs the tests under `__tests__/content` on PGlite as the real app role; UI tests run outside CI.

- Limits: I have not measured the share of 'awaiting review' proposals that staff applied, i.e. proposal accuracy. The server does not recheck evidence phrases; it trusts the checks on the intake side.

- If I did it again: I would ship a test that calls each new DB function as the app role. The 107 rejected summaries on 10-23 would have been caught before deploy by that one test.

What I decided

- Counterpart matching uses an ordered rule list instead of an LLM. When keys disagree, it does not match automatically and hands the record to staff.

- Automatic stage updates are turned on one at a time through a (source, stage) rule table. The first one enabled is KakaoTalk shipment·order·group buy → 'in progress'. LLM self-reported confidence is not calibrated, so it is not used.

- A stage that staff rejected is never proposed again. So the review queue holds only proposals nobody has judged yet.

Vendor CS

Vendor CS Agent

StyleSeller · private company repositories

A KakaoTalk chatbot that answers only tagged questions, never hands off, and rewrites any number or brand that does not match the evidence

Period	2026.06–present
Role	Design: intake path, counterpart gate, evidence-matching rules / Implementation: server agent, send and delivery confirmation in a kakaocli fork (a CLI that reads and sends via the Mac KakaoTalk DB), Windows collector / Operations: incident response, the 09-02 switch in answering mode. I used AI coding tools; I set the scope and checked the results with tests and measurements
Repositories	agent-company (vendor agent), kakaocli fork, CS Copilot extension and local sender

Background

KakaoTalk has no official API, so a server alone cannot read or send counterpart conversations. Staff PCs collect the conversations and upload them to the server, and replies also go out from staff Macs. The server agent writes answers by searching per-counterpart records and the LightRAG knowledge hub (graph RAG over entities and relations, 13 workspaces). Since 2026-09-02 it no longer asks follow-up questions and answers only tagged questions, with no handoff. In exchange, it checks amounts, percentages, and brands against the evidence, and if they do not match it searches again and rewrites.

Structure and scale

1. Tech stack

macOS kakaocli fork (Swift) · Windows read-only memory collector (Python) · Python 3.12 · Gemini Flash family (via LiteLLM) · LightRAG 1.5.6 · Google Sheets and Drive

2. Scale

- Vendor agent code, tests, and config: 61K lines total, 600 commits, 107 test files (2026-09-23)
- CS Copilot extension and local sender: 631 commits, 20,596 lines, 56 tests (2026-09-23)
- Tests: counterpart gate 20, evidence judgment 16, liveness markers 30, Mac Swift 34, Windows Python 72

3. How it works

① On Mac, kakaocli reads only what comes after the last logId in the KakaoTalk DB. On Windows, a collector that only reads memory uploads it. The position advances only after the server answers with 2xx.

② The server drops the same message if it arrives again within 15 seconds. It stores records in per-counterpart ledgers, Sheets, and Drive, and twice a day (09:30, 18:10) compiles them into a wiki that goes into the knowledge hub.

③ The counterpart ID or name in the room name is checked against the brand-tool list of counterparts to serve. If no counterpart is found on the list, it does not answer.

④ It accepts only questions tagged @스타일셀러_AI. The regex only detects the tag; the model does the search and writes the answer. Other counterparts' names are masked as [다른 벤더].

⑤ Amounts, percentages, and brands in the draft are checked against the evidence, and on a mismatch it searches again and rewrites. The reply is sent from a staff Mac, and delivery is confirmed by whether the KakaoTalk DB count went up.

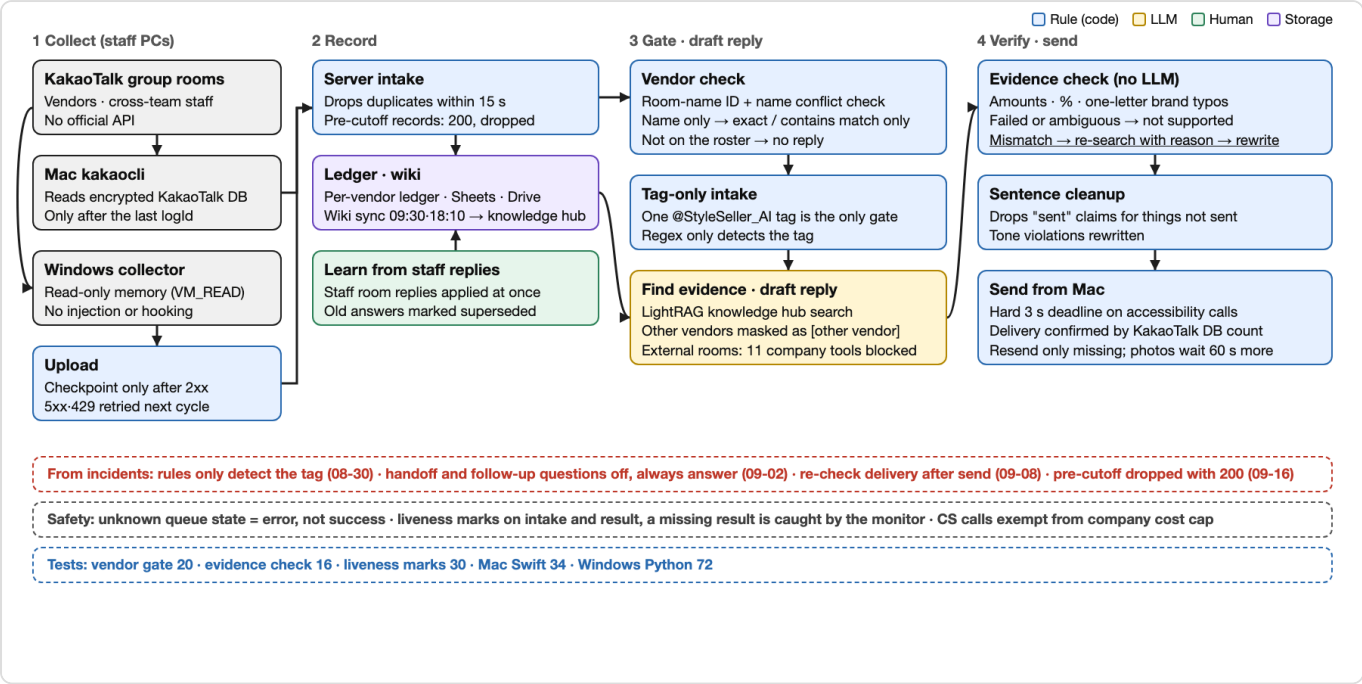
Incidents and fixes

1. **Rules hijacked the judgment (until 2026-08-30).** Some messages were silently blocked without any LLM judgment. The regex meant as a trigger was also doing the blocking. I cut the regex down to tag detection only and moved the decision on whether to answer to the model.

2. **Sends never finished (2026-08-17–18).** On macOS 15, when the KakaoTalk window was in the background, Accessibility (AX) calls ignored their timeout and hung. 31 of 147 sends (21%) ended as permanent failures. I moved the calls to a separate thread and put a 3-second deadline on them with a semaphore (past that, the wait is cut off). Since 09-08 it also re-checks delivery after sending.

3. **Flood of past conversations (2026-09-16).** Three Windows PCs uploaded 2,500 records in 50 minutes, the Sheets API was blocked with 429, and rows for the 2 people installed later stayed empty for over an hour. An old collector build had uploaded entire past conversations. I made the server drop records older than a cutoff date and answer 200. Answering with 5xx makes the collector upload the same records again.

Architecture



Staff PC intake → server ledger and wiki → knowledge hub → answer drafted for tagged question → evidence check and rewrite → Mac send and delivery confirmation

Highlights

1. **A similar name does not decide the counterpart.** The '(2670)' at the start of a room name is read as the ID. If the name part points better to a different vendor, at jamo similarity 0.80 or higher, it is treated as an ID typo and blocked. Without an ID, only a single name that is an exact match or a containment match is accepted. Sharing just a common suffix already scored 0.81–0.88, so I dropped fuzzy matching.

2. **One message came in 51 times.** kakaocli resends the latest message of an unread room every 2 seconds. When I counted, it was 51 times for 1 message, 170–200 per minute. A (room, message, unread count, sent by me) key drops resends within 15 seconds. Because the unread count is in the key, the signal from closing and reopening a room still gets through.

3. **Numbers and brands are checked in code.** Amounts and percentages in the draft are normalized ('3만원' is 30000원). Values not in the retrieved evidence, and words one character off from an evidence brand, are flagged as violations. If the LLM judgment fails or is ambiguous, it counts as no evidence. When a violation is found, it searches again with the reason attached and rewrites.

4. **'Sent' is written only when it is true.** If the send API was not called in that turn, completed-action sentences like '제안서 보내드렸습니다' ("I've sent the proposal") are removed. When a retry arrives with the same idempotency key, it checks the actual queue state. Items in progress or already sent are not re-queued; only failed ones are retried. If the state is unknown, it returns an error instead of success.

5. **Other counterparts' cases are masked.** If a vendor name other than the current counterpart appears in search results, it is replaced with [다른 벤더], and the model is told that the replacement happened. All 11 Spark tools that query company-wide data are blocked in external rooms. The vendor name passed to the product recommendation lookup is set by the server, not the model.

6. **Delivery is counted from the KakaoTalk DB.** A baseline count is taken before sending. After sending, it counts how many text messages and photos were added in the local DB and resends only what is missing. Photos get an extra 60 seconds. In a 70-person room, the same photo arriving twice is worse than one photo missing. Completion is reported to the server only after this check.

7. **The checkpoint moves only after 2xx.** The Windows collector only reads KakaoTalk process memory with VM_READ access; it does no injection or hooking. The uploader moves the offset only after receiving 2xx. 422 and 413 are skipped; on 5xx or 429 it stops without committing and uploads again on the next run.

8. **The processing path leaves its own liveness markers.** It leaves one marker when a tagged message is accepted and one when it reaches a result. If no result marker appears past a business-hours threshold, the monitor catches it. The earlier approach, where the monitor guessed from outside that 'no draft means it stalled,' was removed in review. This path has 30 tests.

Status, judgment, and limits

- Since 2026.06, intake, ledger and wiki compilation, and answers to tagged questions have been running. Replies that staff send directly in a room are learned right away, and older answers to the same question are marked as superseded by the new one and demoted in search.

- Measured: the resend was 51 requests for 1 real message, and during the AX hang 31 of 147 sends failed. Answer accuracy and rewrite rate after the 09-02 switch were not measured.

- Vendor and influencer CS calls are excluded from the company daily cost cap. The Windows collector has no auto-updater, so new builds are deployed by reinstalling.

- If I did it again: I would put auto-update and the cutoff date into the first build of the Windows collector. The 09-16 flood of past conversations came from an old collector build, and there was no way to fix it except reinstalling.

What I decided

- On 2026-09-02 I turned off follow-up questions and changed it to answer only tagged questions, with no handoff. In exchange, number and brand checks are done in code, not left to the LLM. The old handoff branch stays in the code, guarded by tests.

- The regex handles only tag detection, and judgment went to the model. The approach where the regex also did the blocking was removed on 2026-08-30.

- I took fuzzy matching out of counterpart checks. Not answering is better than answering in someone else's room because a name looked similar.

Work Tool

Internal Work Tool · Pricing, Matching, Personalized KakaoTalk

StyleSeller · private company repositories

Code computes the amounts; the LLM only returns candidates and the fields to change

Period	Late 2026.03 – present
Role	Design: pricing calculation structure, rules for merging match candidates, send-approval criteria / Implementation: pricing code, catalog extraction, matching, send automation, PC worker / Verification & operations: sheet-comparison tests, running sends and workers (seller recommendation logic is a colleague's CRM)
Repositories	brand-tool, agent-company (staff PC worker)

Background

Pricing lived in Google Sheets formulas, and every proposal was reconciled by hand. When I moved those formulas into code, I added tests that compare each field against the sheet values for real products. In catalog product extraction and sales partner matching, the LLM only produces candidates. Registering and reviewing them is the staff's job. For personalized KakaoTalk, a CRM built by a colleague decides each seller's recommendations; I built the send automation that takes those targets and sends.

Structure and scale

1. Tech stack

Next.js 16.2·React 19.2·Tailwind 4·Vitest 4 · Postgres+PostgREST (a REST server that exposes tables directly) · Gemini Flash family (via LiteLLM) · Knowledge hub RAG search (finds material and attaches it to the answer) · kakaocli · Staff PC worker

2. Scale

- Personalized KakaoTalk sent to 261 people in one day (2026-09-22)
- 43 pricing test cases, some compared field by field against sheet values for real products
- Seller-finder worker on 9 staff PCs (2026-09-18)

3. How it works

① Pricing: I ported the sheet formulas to code, down to the rounding units. When an edit request comes in, the LLM only emits a JSON of input values to change, and code recomputes the amounts.

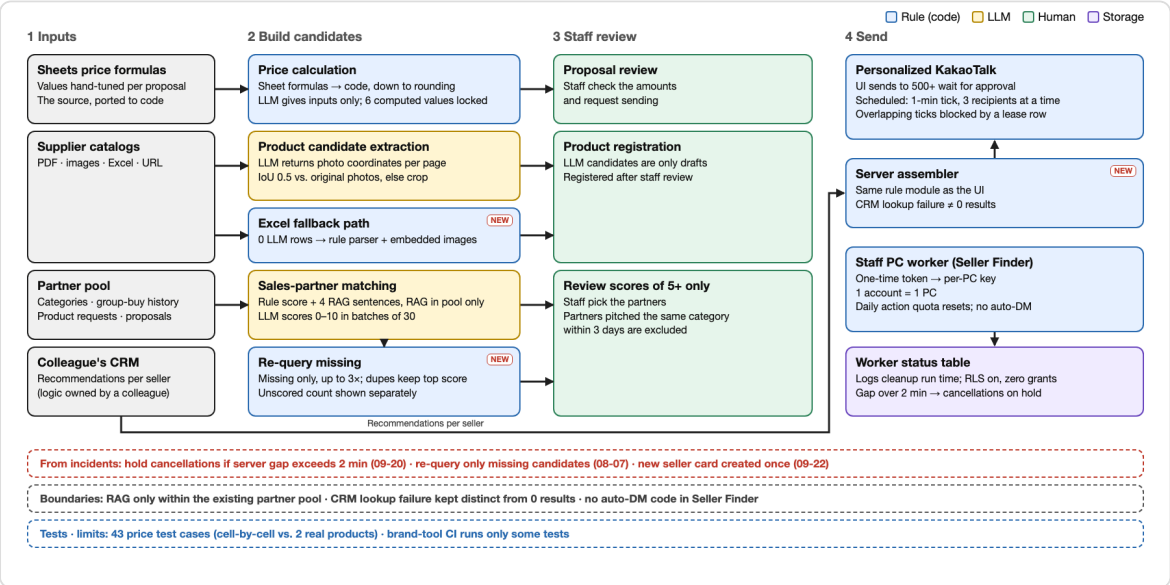
② Catalog: the LLM extracts product candidates from PDFs, images, Excel files, and URLs. They are registered only after staff confirm them.

③ Matching: rule-score candidates and RAG candidates are merged, and the LLM scores them 0–10 in batches of 30. Staff see only those scoring 5 or higher, and anyone pitched in the same category within 3 days is excluded.

④ Personalized KakaoTalk: operators send, schedule, and pause from the UI. Sends to 500 or more people go to an approval queue, and scheduled sends go out a little at a time, every minute, from the server.

⑤ Seller finder: the staff PC worker runs one PC per account, within a daily action cap. I did not include any auto-DM code.

Architecture



Sheet formulas→code calculation · Catalog & partner pool→LLM candidates→staff confirmation · CRM targets→server assembly→send / approval queue

Incidents and fixes

1. **Missing match candidates (2026-07-15).** In 3 of 6 chunks (the batches sent to the LLM separately), the LLM ended its response before judging all candidates. Since 07-16, candidates are collected as a union and then split for querying, and since 08-07 only the missing candidates are re-queried, up to 3 times. The count that still goes unjudged is shown separately in the UI.

2. **Shared worker key (2026-09-10).** Every staff PC worker was sharing a single key. I changed it so each PC gets its own key through a 30-minute one-time install token. The server stores only the key's sha256, and comparison uses timingSafeEqual.

3. **Six runs cancelled at once (2026-09-20).** When the server came back after being down for two days, 6 in-progress runs were cancelled at once. The 'worker not seen for 3 minutes' rule read the server gap as workers dropping out. If the interval between cleanup runs exceeds 2 minutes, it is now treated as a server gap and the reference time is reset.

Highlights

1. **I defined which fields the LLM may change.** If the LLM back-calculates a request like "set seller earnings to 1500", other amounts shift too. So the 6 computed values, such as supply price and earnings, are blocked from the edit list. For such requests, it only points to the input values that need to change. If asked to change only the margin rate, it does not return the sale price.

2. **I ported the sheet formulas down to the rounding units.** Partner supply price rounds up to the nearest 10 won, and the payment fee is 3.96%. I added tests that compare each field for 2 real products in the sheet (BAWANG at 30,190 won, among others). The tests also check two properties: changing the split ratio leaves total profit unchanged, and the two shares always sum to total profit.

3. **It looks for the original photo inside the PDF first.** Pages go to the LLM one at a time, and it returns photo coordinates. If those coordinates overlap an original image embedded in the PDF with IoU (the ratio of the overlapping area of two boxes) above 0.5, the original is used. Backgrounds covering more than 80% of the page are dropped from the candidates, and the page is cropped only when no matching original exists. For products that appear on several pages, empty price fields are filled from each other.

4. **Excel still yields results when the LLM fails.** If LLM parsing returns 0 items, the UI does not show an empty screen; a rule-based parser re-reads the same file. Images inside the file are attached to products in order. Icons under 750 bytes are not counted as product photos and are discarded.

5. **RAG can only add candidates, not create new ones.** Rule scores are assigned by category, group-buy history, product requests, and proposals, in that order, and the results of a 4-sentence RAG query are added as a union. IDs returned by RAG are accepted only if they are in the existing partner pool. This logic used to live in n8n; I moved it into code so it could be tested with vitest.

6. **Automatic sends and UI sends use the same rules.** The server assembler loads the same product-selection and template modules as the UI, so the same person never gets different products. A CRM lookup failure and 0 results are handled separately. Only on 0 results does it fill in popular products. The new-seller card is built only once. On 09-22, when a card was built for each person, the renderer threw 980 500 errors.

7. **Scheduled sends go out a little every minute, without overlapping.** Per tick, it assembles up to 3 targets that get personalized recommendations and up to 30 without them. If 5 or more queued items have not gone out, that tick is skipped. Once the send window passes, nothing new starts, so sends do not pile up at night. To keep ticks from overlapping, each schedule has a lock row, and it can only be released with the token left when it was acquired.

8. **One account runs on only one PC.** Checking the install token and marking it used are combined into a single conditional UPDATE, so two PCs cannot register with the same token at the same time. If the same Instagram account is on two PCs, the one that logged in more recently wins. The server's job-assignment function follows the same rule. The daily action volume is redrawn within a range every morning.

Status, judgment, and limits

- In operation. Personalized KakaoTalk is sent from the UI (261 people on 2026-09-22), and scheduled sends go out from the server every minute. Per-slot on/off values live in the DB.

- Judgment call: after a server gap, job cancellation waits 3 more minutes. I judged that being 3 minutes late is cheaper than cancelling by mistake.

- Limits: brand-tool CI runs only some of the tests. I have not measured how well match scores agree with what staff actually pick.

- If I did it again: I would make CI run the full test suite and make the 43 pricing test cases a deploy gate.

What I decided

- Amounts are not left to the LLM. Code ported from the sheet computes them, and the LLM only returns input values to change.

- RAG search is used only to widen the candidate set. Companies outside the existing partner pool cannot get in.

- Sends to 500 or more people are not sent immediately; they wait in the approval queue.

Knowledge Hub

LightRAG Knowledge Hub · Email Agent

StyleSeller · private company repositories

Four agents pull evidence from one knowledge hub, and only evidence-backed email drafts reach staff drafts folders

Period	2026.03.31~present
Role	Design: workspace registry, search-planning and evidence-judgment stages, draft verification order / Implementation: email-agent server, checkers, sent-mail learning cursor / Operations: drafts-folder rollout, incident analysis and recovery
Repositories	email-agent

Background

Vendor CS, influencer CS, Spark, and sales partner matching share one LightRAG knowledge hub (13 workspaces). It is a RAG store: for each question it finds material first and answers with it attached. Every workspace carries a description, owner, channel, and sensitivity level, and a write to an unknown workspace fails immediately. For inquiries to the shared mailbox, it puts an evidence-backed reply draft in the owner's Gmail drafts folder. Staff do the sending.

Structure and scale

1. Tech stack

Python 3.12·FastAPI·Pydantic v2 · LightRAG 1.5.6 (Postgres/pgvector + Neo4j 5) · RAG-Anything/MinerU · FlashRank (lightweight rerank model) · Gemini Flash family (via LiteLLM) · Gmail · pytest

2. Scale

- ~1,470 commits · ~47,000 lines of product code · ~33,600 lines of tests (10-24)
- 27 routers · ~1,750 test functions · 13 workspaces (10-24)
- Search eval set of 18 cases (query, expected workspace, required evidence, forbidden claims)

3. How it works

① An LLM planner (temperature 0) reads the workspace descriptions and data catalog and builds up to 12 queries. A plan that fails validation is discarded and the rule router takes over.

② Code pins the search mode (mix, which uses vector and graph together) at each call site. Agent searches use top_k (number of results fetched) 8, and identical queries are cached for 60 seconds.

③ Evidence is judged by rules. Live values such as price, stock, and settlement are forbidden claims. Refund confirmation, contracts, and bulk sends get a human approval flag, and only the missing evidence items are searched once more.

④ Results are grouped by topic using rerank (the step that re-orders results) scores. Within a group, official corrections and the newest documents go first, and output alternates one result per group.

⑤ A checker and an LLM verifier compare up to 6 claims in the email draft against the evidence. If a claim has no evidence the draft is rewritten, and if the problem remains the draft is discarded.

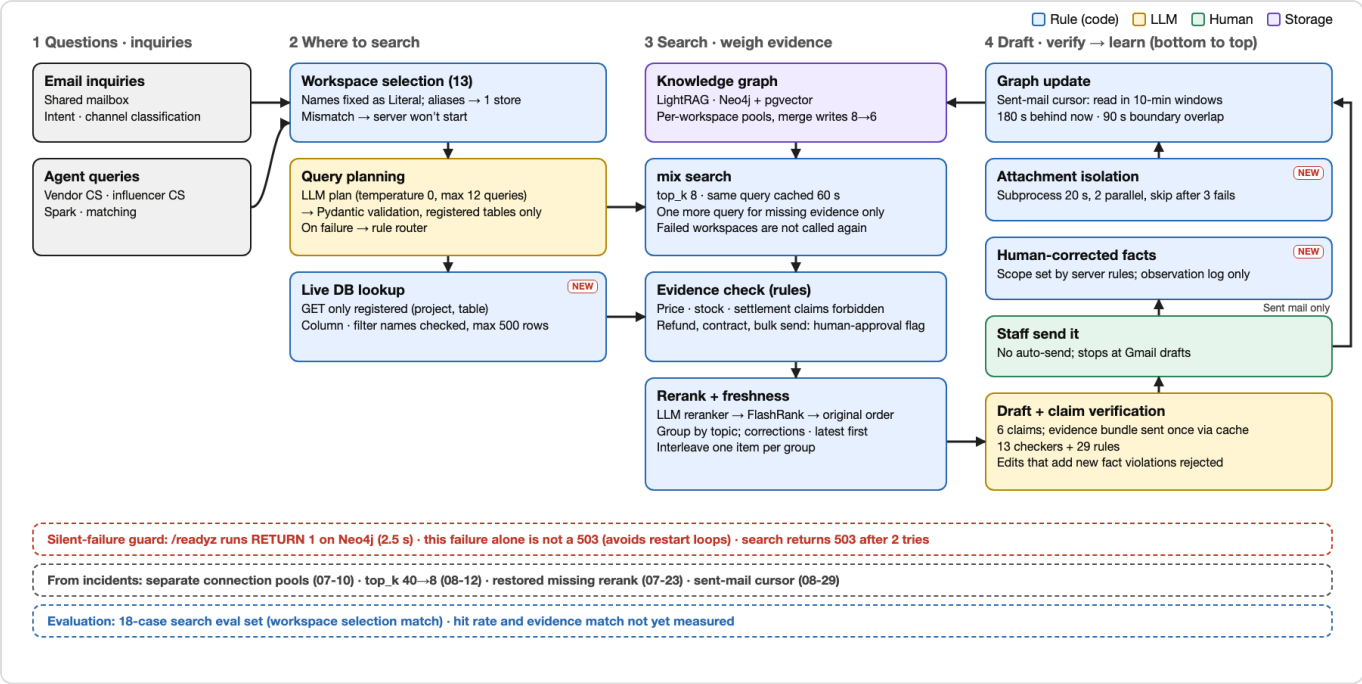
Incidents and fixes

1. **Missing rerank (2026-06-19~07-23).** On 06-18 we switched to agentic RAG (the LLM plans the search). From the next day, the CEO email draft edit rate (share of drafts edited before sending) rose from 37% to 82%. Tracing back through the commit history showed the rerank step had been dropped in the switch. I restored it on 07-23.

2. **Search overload (07-10, 08-12).** The influencer CS bot took the whole shared connection pool, so even matching searches returned 500 (07-10). top_k 40 searches triggered Neo4j overload warnings (08-12 analysis). I split the connection pool per workspace (kakao_cs 20, etc.) and capped agent searches at top_k 8.

3. **Silent graph outage (2026-10-04).** When Neo4j stopped, the container still looked healthy. Now /readyz sends RETURN 1 to Neo4j with a 2.5-second limit and reports ok or fail. This failure alone does not return 503, so it does not cause a restart loop. Search answers with 503 and Retry-After 30 after 2 attempts.

Architecture



Sent-mail facts · KakaoTalk wiki · PDFs → knowledge hub (entities · relations) → LLM planner → search · evidence judgment → drafts · answers

Highlights

1. **The server will not start if workspace names drift.** If each agent calls a workspace by a different name, writes can land in the wrong store. The 13 canonical names are fixed as a Literal type, and ~50 aliases each resolve to one store. If a missing entry or store mismatch shows up at module import, startup halts with a RuntimeError.

2. **An LLM search plan is used only after it passes validation.** Plans are checked with Pydantic. Live DB lookups are allowed only for registered (project, table) pairs, GET only, up to 500 rows. If any one of the call, the parse, or the allowlist check fails, it falls back to the rule router. The reason is logged as `fallback:<exception name>`.

3. **If rerank fails, two more stages stand in.** Documents are cut to 20, then the LLM reranker runs first. If it fails, FlashRank runs; if that fails too, the original order is scored, so results always come back. I lowered search-stage concurrency to 3. That cut concurrent Neo4j writes in the merge stage from 8 to 6 per document.

4. **13 draft checkers, each owning one error pattern.** Patterns include reversing the direction of a request, citing a previous email that never existed, re-asking a confirmed condition, and writing a proposed price as if it were final. Each checker includes reproduction cases from real emails of 08-21~08-23. They started as regexes, but false positives kept coming, so judgment moved to LLM checkers.

5. **A fix that introduces a new falsehood is rejected.** Comparing only the sum of violation scores lets a 2-point factual violation pass as the price of fixing a 10-point forbidden word. So if a rewritten draft has a factual violation that was not there before, the rewrite is discarded even if it fixed other violations. Violations that remain after rewriting are logged as "Draft rejected" by type, and the draft is dropped.

6. **The evidence bundle is sent once.** Each time 6 claims were verified, the evidence bundle of up to 150,000 characters was being resent. The bundle is now cached for 5 minutes under a sha256 key and reused. If the cache fails it falls back to the original path, so verdicts do not change.

7. **The sent-mail cursor accounts for Gmail lag.** At rollout, 48 of 1,943 sent emails were missed by learning. Now it reads in 10-minute windows and saves the cursor after each window. Because the search index picks up mail late, the cursor stays 180 seconds behind now, and window boundaries overlap by 90 seconds. For a window with more than 500 emails, the cursor does not advance.

8. **The server decides the scope of facts staff correct.** Server rules decide whether a corrected fact applies company-wide or to a single deal. A single-deal fact is never widened to the whole partner. Results only accumulate in an append-only observation log, running in shadow mode: not used for drafts or search.

Status, judgment, and limits

- Reply drafts go no further than the owner's Gmail drafts folder. Sending is up to staff, and since 2026-08-18 only images that passed pytest are deployed.

- Attachment extraction runs in a `python -I` child process with a 20-second limit and at most 2 at once. A chunk that fails 3 times is skipped from the next ingest on.

- Limits: the 18 search eval cases check only workspace selection. Search hit rate, evidence match, and the effect after restoring rerank have not been measured.

- If I did it again: I would add evidence-match scoring to the eval set first. That would have caught the missing rerank before the edit rate climbed to 82%.

What I decided

- The LLM planner decides where to search; code decides the search mode. I did not let agents pick the mode.

- The only source of facts is email staff actually sent. AI drafts are never fed back into the knowledge hub.

- Facts corrected by people are not used right away; they go to the observation log first so the scope judgment can be checked.

Platform

Agent Platform · Spark

StyleSeller · private company repositories

No matter who asks, work starts as a work order from the CEO agent, and the server controls delegation and tool execution

Period	2026.05–present (kickoff 05-17, Spark·Work OS design 07-18~19)
Role	Design: work order, delegation guards, tool gate structure. Implementation: Spark SQL, single-use approvals, LiteLLM spend cap, Jev (a router that shortlists skill candidates) integration. Verification and operations: writing eval sets and VM deployment. I used AI coding tools; I set the scope and checked the results myself
Repositories	agent-company (Paperclip fork), agent-skills, services/*

Background

An internal platform where agents hand issues to each other to get work done. I forked Paperclip (an open-source agent issue tracker) and attached company features as HTTP services under services/ instead of putting them in the core. The only exceptions are Work OS (an app for work status, approvals, and run history) and Spark. Staff use Spark to query 4 business DBs read-only, and hand work to the CEO agent. Long delegation chains change the request, and a single tool call can lead to spending money or external exposure. So the controls live in server code, not in prompts.

Structure and scale

1. Tech stack

Paperclip fork (Express 5, Drizzle, PostgreSQL) · MCP SDK 1.29 · Hermes (agent execution runtime) · Python 3.12 and FastAPI · LiteLLM · Redis · Langfuse 4.50 · Gemini Flash family (via LiteLLM) · Jev router

2. Scale

- About 5,000 of my own commits (excluding merges and upstream), about 2,200 test files (2026-10-24)
- 109 shared skills, 66 trigger eval files (1,282 sentences), 67 tool manifests
- Jev rollout at 100% (content, Spark, Hermes, 2026-10-24)

3. How it works

① Requests from Spark, Slack, and the brand tool all go to the CEO agent. Only structured brand-tool requests go straight to the content orchestrator, and they still leave a work order record.

② The CEO writes work order v1. It holds the original text as an attachment with its sha256, the goal, completion criteria, constraints, and whether approval is required.

③ A Hermes dispatcher reporting directly to the CEO assigns it to one of at most 10 department heads. The server caps delegation depth at 2 and delegations per work order at 5, and blocks A→B→A bounce-backs. Turning it off falls back to direct delegation by the CEO.

④ Workers pick from the 109 shared skills themselves and open the skill body. Jev shortlists 12 with BM25 (a keyword search ranking formula), then passes the top 3 as hints only. If it fails, work continues without hints.

⑤ Tools run through a 6-stage gate and human approval. Publishing goes through the Publisher gateway (a single service dedicated to publishing). The default is manual, and the DB enforces one post per account per day.

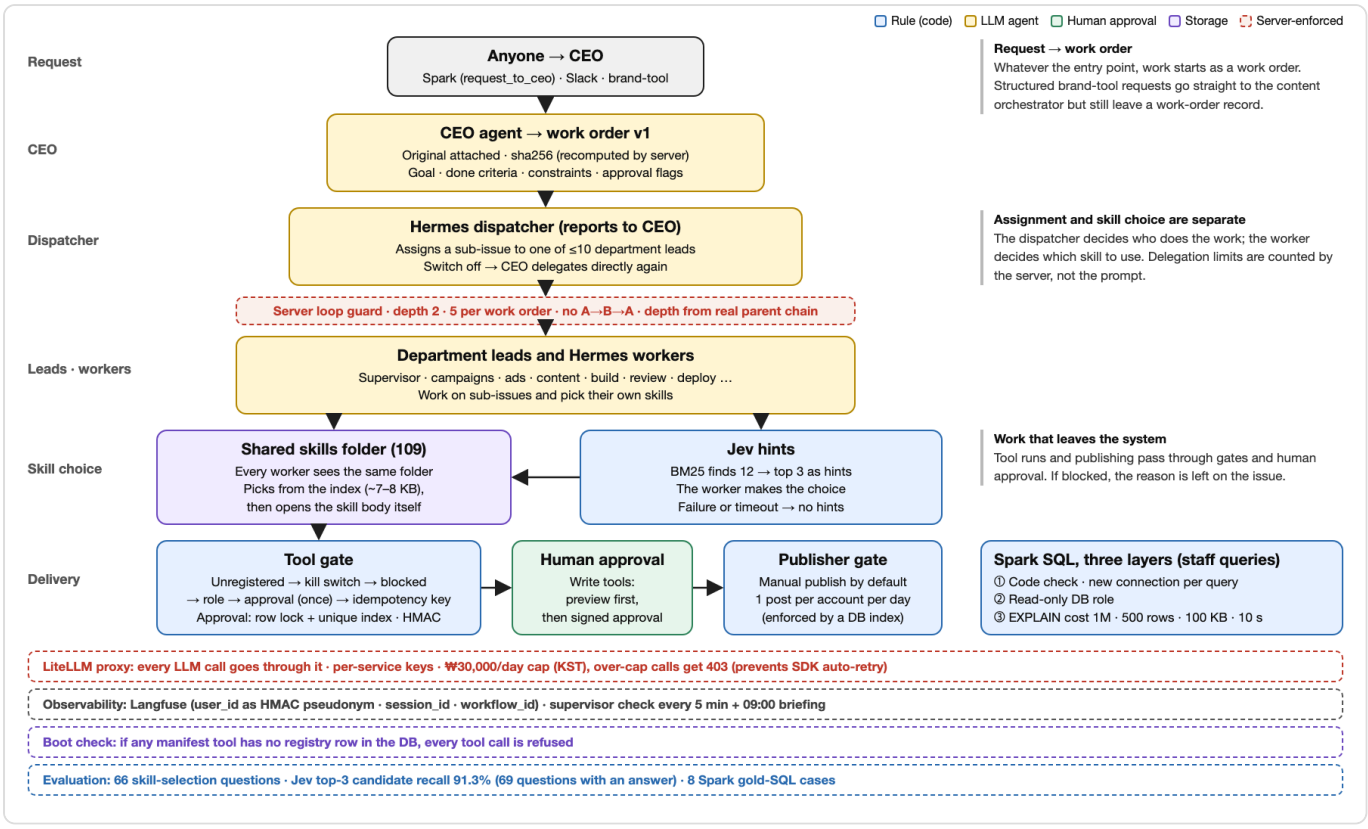
Incidents and fixes

1. **Spark shut down entirely (2026-10-24).** A new Spark tool was missing its DB registration row, so the boot check failed and every Spark call was rejected. I built it to block when the check fails, and it did exactly that. After recovering with a restart, I added a rule that every tool's registration row ships as a migration, plus a regression test.

2. **Reclaiming a job mid-render (2026-10-24, #627).** The server reaper treated a video job running past 5 minutes as stalled and reclaimed it, and the result that arrived 7 min 40 s later was rejected with 409. I made the job send a heartbeat every 120 seconds and stop once the issue deadline passes. Jobs that are really stuck still get reclaimed. In production I confirmed heartbeats received (200) and 0 reclaims.

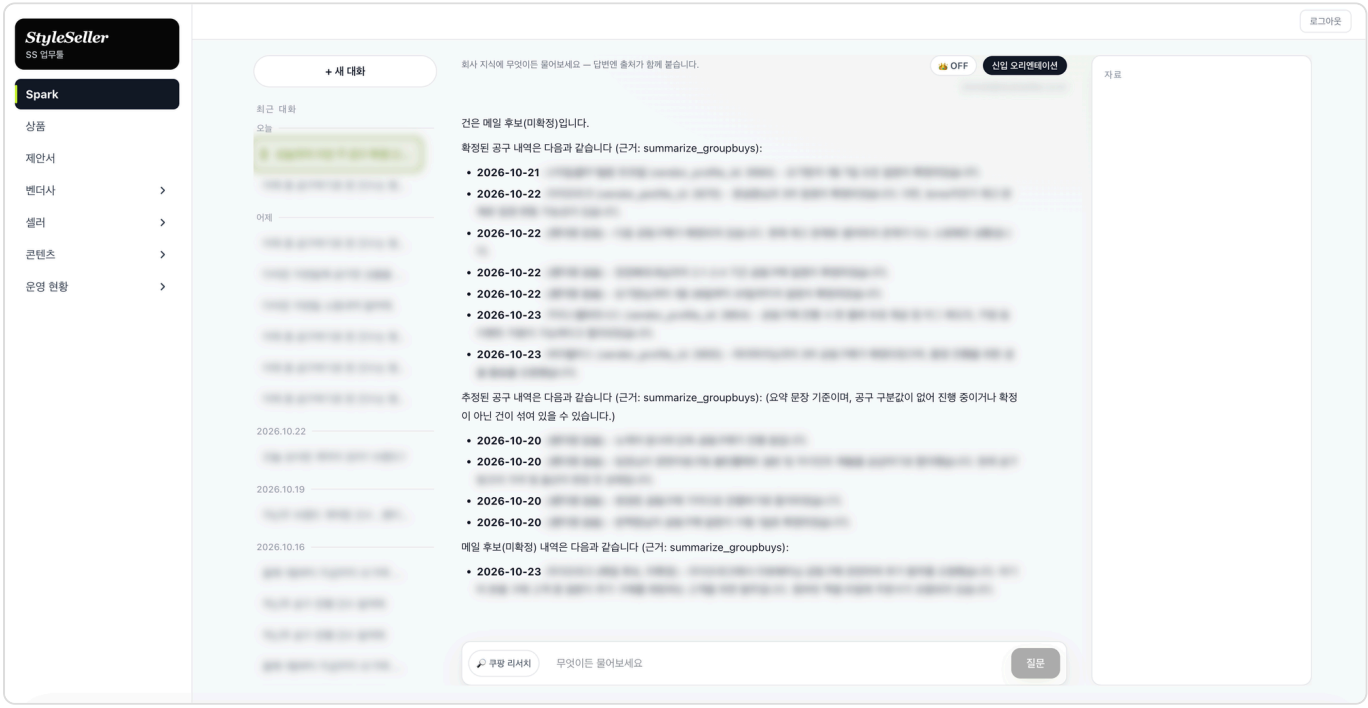
3. **Jev call path blocked (2026-10-23).** The existing Jev call path was blocked. I added an adapter that only matches the response format and sent calls to a lightweight model. The code in the 3 routers stayed the same. Past the free quota, it does not fall over to paid; it proceeds without hints.

Architecture



Request → CEO work order → Hermes dispatcher → department heads·workers (Jev hints) → tool gate·human approval → Publisher; every LLM call goes through LiteLLM

Screen



Spark answer screen. Question titles and counterpart names are blurred; each answer shows the tool it relied on (summarize_groupbuys).

Highlights

1. **The work order's original text is locked with a hash.** Each delegation step changes the wording of the request. The original is NFC-normalized, line endings are normalized, and it is locked together with its sha256. The server overwrites any hash the client sends. I followed an external benchmark that found fewer supervisor errors when the original is passed along instead of an intermediate summary.

2. **The server breaks delegation loops.** Plugins can only observe after an issue is created, so I changed the server core. Depth 2, 5 per work order, no A→B→A. Depth is counted by walking up the actual parent issues, not read from a column whose value can be changed. A 6th delegation and forged depth are rejected in tests.

3. **Jev gives hints; it does not choose.** It shortlists 12 with BM25, compares the top 3, and inserts them as hints. The worker makes the choice. On failure, timeout, or an open circuit breaker (300 seconds after 2 failures), the whole hint block is dropped. Rollout targets are split by the first 8 characters of a sha256.

4. **An approval is used exactly once.** The approval row is locked with FOR UPDATE, and a partial unique index blocks it a second time. The approval is consumed even if execution fails. The idempotency key uses ON CONFLICT DO NOTHING followed by a re-read, so there is no race. A test confirmed 1 execution for 10 concurrent requests.

5. **Write approvals sign the preview.** Input hash, preview hash, requester, run ID, and expiry are bundled as canonical JSON and signed with HMAC-SHA256. Write access to the DB is not enough to pass without the key. The key contains no timestamp, so model retries do not multiply approvals.

6. **Spark SQL opens a new connection per query.** A read-only role alone was not enough. One set_config call lifts the timeout and read-only mode, and a reused connection hands that state to the next person. After filtering 23 dangerous functions, semicolons, and writing CTEs, it opens 1 connection and closes it when done.

7. **Expensive queries are rejected before they run.** If the EXPLAIN cost exceeds 1,000,000, the query does not run. The 10-second limit is set in two places: the DB setting and a socket close in the app. Results are cut at 500 rows or 100KB. The role has BYPASSRLS so RLS cannot silently empty results, and the audit log keeps only the SQL hash.

8. **Exceeding the paid cap returns 403, not 429.** OpenAI-family SDKs retry 429 and 5xx on their own. So once spend passes 30,000 KRW a day, it returns 403 with x-should-retry: false. The ledger is a Redis key per KST date. Service keys can call only the inference path, and overlapping keys are discarded.

Status, judgment, and limits

- In production. The company-wide tool execution switch is off by default, and each tool is turned on separately. user_id is sent to Langfuse as an HMAC pseudonym.

- Evaluation (with the model in use at measurement time): on 69 questions with known answers, Jev candidate recall (share of cases where the answer is among the candidates) is 85.5% at top 1 and 91.3% at top 3 (2026-10-24). Median latency is about 1.09 seconds.

- Limits: evals run manually, not in CI, and the Spark eval has only 8 cases with reference SQL. I did not measure whether Jev hints actually changed task outcomes.

- If I did it again: I would check manifests against tool registration rows in CI. The 10-24 outage only surfaced at boot, but it could have been caught before merge.

What I decided

- I chose a full index over per-role skill bundles. On a 57-case selection set both scored the same accuracy, 0.947 (p=1.0). But the bundle approach missed skills that only get used when called by name. The cost difference over 66 cases was 27 KRW.

- The dispatcher assigns work; the worker picks the skill. I did not make Jev the selector, because work must not stop when Jev stops.

- Controls sit where they still hold if the model ignores them. The loop guard, single-use approval, and paid cap all live in the server and DB.

Content

Content Autopilot

StyleSeller · private company repositories

From a Reels request to a 9:16 MP4 render and approval, all in the operator's screen. Live SNS posting is simulated

Period	2026.10.03–present
Role	Design: the stage state machine and posting defaults. Implementation: the orchestrator, the publish gate and DB slots, the QA gate. Verification: a production canary (running a single item first as a trial) and restart-scenario tests. I used AI coding tools; I set the scope and checked the results myself
Repositories	content-orchestrator, hermes-content-agent, video-studio, brand-tool(/sscrew/contents)

Background

Staff request a Reel in the work tool at /sscrew/contents, review the preview, approve it, and schedule the release. Generation costs money, and posting means public exposure. So every default is set to 'don't do it'. Posting is manual, recurring automations run as dry-run (they only simulate execution), and paid generation is off. Code drives the stage progression.

Structure and scale

1. Tech stack

Paperclip sub-issues · content-orchestrator (state machine with no LLM) · Hermes and the Gemini Flash family (via LiteLLM) · image-service · TTS · video-studio · HyperFrames (a tool that renders video from HTML) · ffprobe · Google Drive · Blotato (scheduled SNS posting service) · Supabase (PostgreSQL)

2. Scale

- About 120k lines across 3 services, about 300 test files
- Production canary verified through saving a 1080×1920 MP4 to Drive (2026-10)
- 0 live SNS posts. One post per account per day is enforced by the DB; the publish gate caps at 10 per day (2026-10-24)

3. How it works

① Staff request a Reel, or run a saved automation (immediate, one-time scheduled, or recurring).

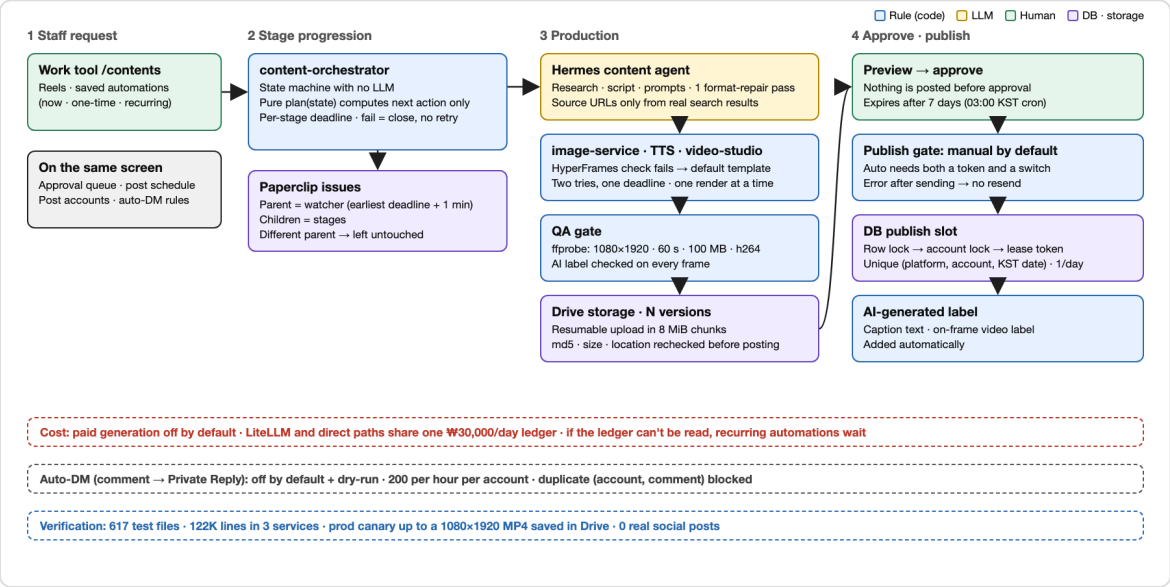
② A pure function in the orchestrator computes only the next action. An executor applies it to the Paperclip sub-issues to advance the stage.

③ A Hermes agent writes the research, script, and prompts. video-studio renders a 9:16 MP4 from a HyperFrames template.

④ Versions that pass the QA gate are saved to Drive with a checksum. A human approves from the preview; anything pending for more than 7 days expires.

⑤ Posting is manual by default. Staff download the MP4, copy the caption, and click 'Posted'. The 'AI-generated content' label and the frame marker are added automatically.

Architecture



Request → state machine (Paperclip issues) → Hermes script → render & QA → human approval → publish gate (manual by default, DB slot)

What the defaults block

1. **Posting.** The default is manual. Auto-posting turns on only when both the token and the switch are present. A partial unique index on (platform, account, KST date) blocks more than 1 post per account per day. If duplicates already exist, the migration stops first.

2. **Paid generation and recurring automations.** The paid backend defaults to False. Recurring automations default to dry-run and are released only when the value is exactly "false". Before starting, they read the company's ₩30,000-per-day ledger, and hold if it can't be read. The cap is clamped with min() so env can't raise it.

3. **Instagram auto-DM.** By default the enable switch is off and dry-run is on. The limit is 200 per account per hour, and invalid config values are read toward the safe side. A unique index on (account, comment) stops a single comment from getting two DMs.

Highlights

1. **A pure function decides the next action.** plan(state) returns only a list of actions, and the executor applies them. Each stage has a deadline, and the parent issue wakes at the earliest deadline + 1 minute to check. On failure, it cancels and closes the sub-issue with no retry. It doesn't touch sub-issues that belong to another parent. So a rerun doesn't create a new pipeline.

2. **Format errors get exactly one repair.** Only JSON format errors are eligible for repair. The previous output isn't resent; only the error list (up to 8 items, 120 chars each) is. At most 2 calls. A test pins down that if it's still wrong after the repair, it fails with no third call.

3. **Only real search results count as sources.** A cited URL passes only if it appears in web tool results or the input. Evidence fields the model wrote itself are discarded. Claiming a search without one fails with web_research_unverified. Facts like product prices are still guarded only by prompt rules.

4. **A post with an unknown outcome is never resent.** A start marker is written right before the POST. If an error occurs after sending, the item is left as publish_pending instead of failed. On restart, if it was already posted, it's closed as already_published. For Blotato, the submission ID is saved immediately, so a later wake-up only queries the status.

5. **The DB hands out publish slots.** The claim function locks the row with FOR UPDATE, takes an account advisory lock, checks the daily cap, then issues a 30–900 second lease token (a lock marker borrowed for a short time). I didn't go with an in-memory app lock. However many processes run, the decision happens in one place: the DB.

6. **One ledger enforces two policies.** LiteLLM and the direct Vertex path add costs to the same Redis key. That closed the gap where the direct path bypassed the proxy cap. If the ledger can't be read, individual calls pass and recurring automation starts are held. Reads use a Redis user with no write permission.

7. **Render retries share one deadline.** If the LLM-written motion plan fails the HyperFrames check, it re-renders with the default template. Both attempts share one deadline, so a retry doesn't double the wait time. Renders run one at a time.

8. **The file to post is pinned by checksum.** Right before posting, the Drive file's mime, size, md5, and location are checked again. The QA gate uses ffprobe to check 60 seconds, 1080×1920, 100MB, and h264, and for AI video it looks for the AI marker in every frame. Uploads read 8MiB at a time and resume on a 308 response (the resume signal).

Status, judgment, and limits

- This is a simulation. There are no live SNS posts; the production canary confirmed up to saving the MP4 to Drive (2026-10).

- Approval expiry is handled by a daily 03:00 KST cron in this order: reject, cancel the issue, close the row. Even if the issue cancel fails, the row is closed so the same failure doesn't repeat.

- Limits: product fact checking is only prompt rules, and registering a new Instagram token needs developer setup. With no live posts, results like view counts haven't been measured.

- If I did it again: I'd add a code check that compares product facts such as price and bundle contents against the product DB. Right now, if the model breaks a rule, a human has to catch it in the preview.

What I decided

- A state machine with no LLM drives stage progression. The LLM only writes research, scripts, and prompts. If the model picked the next stage, every rerun would give a different result.

- On failure, it closes instead of retrying automatically. Anything that costs money or goes public, a human clicks again.

- Posting defaults to manual, and the daily cap lives in a DB index, not in app config.

PC Fleet

Staff PC Deployment · Operations

StyleSeller · private company repositories

KakaoTalk and Instagram work has no official API, so I ship it to 11 staff PCs with a one-line install and run it there (as registered on 2026-09-16)

Period	2026.02–present (Instagram DM extension from 02, KakaoTalk tools from 05)
Role	Design: install, auth, and update structure; server cutoff date (messages before it are dropped) / Build: installers, workers, Windows collector, fixes to the kakaocli fork / Operate: deploys, incident response, monitoring crons. I used AI coding tools; I set the scope and checked the results myself
Repositories	agent-company, kakaocli fork

Background

KakaoTalk and Instagram have no official API a server can call. So the work had to run on staff Macs and Windows PCs. Install is a one-line command, auth is a per-device key, and updates run on an automatic schedule per tool. I also run the servers myself. On 2026-08-20 a cloud DB quota stopped every work-tool API; I stood up the same API on a VM and restored service that day.

Structure and scale

1. Tech stack

macOS bash and Windows PowerShell installers · launchd and Task Scheduler · Node workers · Python memory collector (Windows) · kakaocli fork (Swift) · VM Postgres+PostgREST+nginx · SSE (server-pushed events) push hub · Docker deploy scripts · Slack monitoring crons

2. Scale

- 11 staff PCs (as registered on 2026-09-16), 9 seller-finder worker PCs (09-18)
- 4 DBs moved to self-hosting (2026-08-06~08-25)
- Full work-tool API outage restored the same day (2026-08-20)

3. How it works

① Staff install with a one-line command. Mac uses bash, Windows uses PowerShell, and most installs need no admin rights.

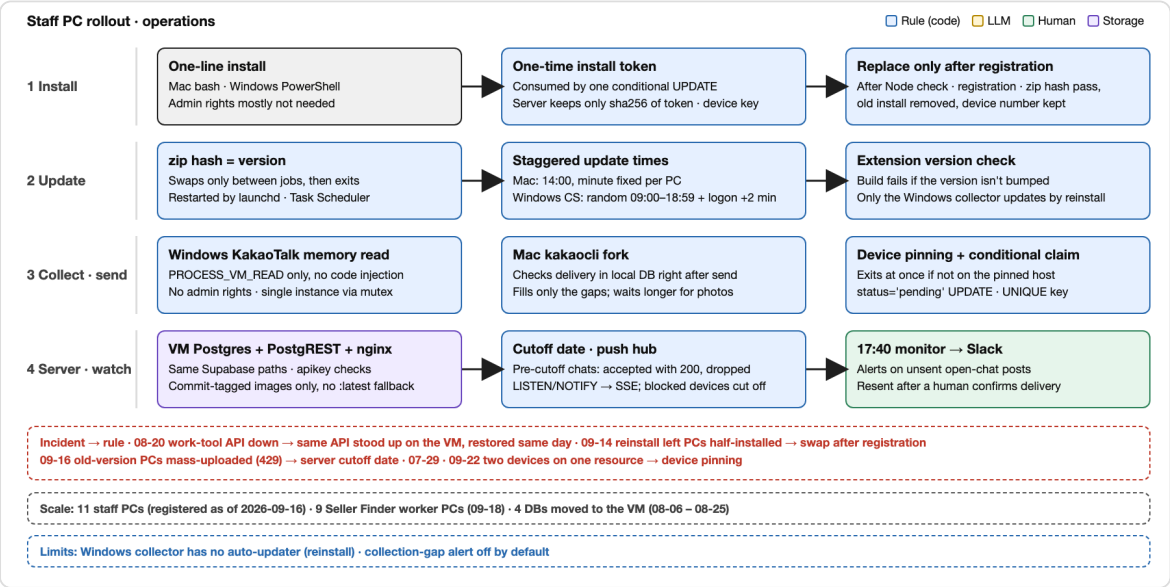
② Registering with a one-time install token returns a per-device key. The server keeps only the sha256 of the token and the key.

③ Each tool has its own auto-update interval, and update times are spread across PCs. Only the Windows collector is replaced by reinstalling.

④ Queue jobs are picked up only by the assigned device. A pickup is a single conditional UPDATE, so two devices cannot grab the same job.

⑤ Servers deploy only from committed images. A 17:40 monitor posts unsent open-chat messages to Slack, and a person checks them before resending.

Architecture



Staff Macs and Windows PCs (one-line install, per-device keys, spread-out auto-updates) ↔ VM (PostgREST+nginx, cutoff date, committed-image deploys) ↔ monitoring crons and Slack

Incidents and fixes

1. **Full work-tool API outage (2026-08-20).** The cloud DB hit its egress quota and every work-tool API stopped. I had nginx on the VM mimic the Supabase paths and apikey header check exactly. The apps changed only their connection settings and a few adapters; the 100-odd createClient calls in app code stayed as they were, and service was back that day.

2. **Reinstall broke healthy PCs (2026-09-14).** The installer deleted the old install first and registered after. When it ran with an expired code, the PC was left with no config and no package. Now the old folder is deleted only after the Node check, registration, and zip hash check all pass. The device number stays the same across reinstalls.

3. **Old-version PCs uploaded their whole chat history (2026-09-16).** Three Windows PCs sent their backlog of chats all at once. The Sheets API blocked with 429, and rows for the 2 staff who installed later stayed empty for over an hour. Fixing the client does nothing until the PC picks up the new version. So the server holds a cutoff date, and messages before it are accepted with 200 and then dropped.

Highlights

1. **An install token is consumed in a single UPDATE.** If lookup and mark-as-used are separate, two PCs can register at once with one token. Registration passes only when an UPDATE conditioned on `used_at IS NULL` and expiry returns a row. Failure reasons are split into sentences staff can read. If one device is compromised, only its row is revoked.

2. **zip hashes instead of version numbers.** The sha256 of the published zip is the version. There is no separate version ledger, so the ledger and the file cannot drift apart. Workers swap only between jobs and simply exit. Restarting is the job of launchd and Task Scheduler. If a worker relaunched itself and two came up, the Instagram profile would break.

3. **Update times differ per PC.** If every device updates in the same minute, there is no window to pull a bad release. Mac real-time sync picks only a random minute within 14:00, and keeps that minute across reinstalls. The Windows CS tool downloads at a random time between 09:00 and 18:59, and tries once more 2 minutes after logon.

4. **On Windows, KakaoTalk memory is only read.** The KakaoTalk 26.8 update moved frida's (an app hooking tool) hook points, and the DB key could not be recovered either (measured on 2026-09-15). Messages were still in memory as plaintext JSON. The collector opens only `PROCESS_VM_READ`. With no injection, KakaoTalk does not crash and no admin rights are needed.

5. **Send success is checked in the DB, not by exit code.** On 2026-09-07 only the photo went out, the text was missing, and the job was still marked `sent`. Now it reads the local KakaoTalk DB right after sending and fills in only the missing part, once, on the spot. If the photo does not show up, it waits longer in case the upload is slow. Two copies of a photo in a 70-person room is worse than a lost message.

6. **Device assignment and conditional claims are layered.** On 2026-07-29 a dev Mac picked up the queue of a Mac reserved for remote work. Successes and failures alternated, and finding the cause took a day. Now a worker exits immediately if it is not the assigned host. A claim is a conditional UPDATE on `status='pending'`, and the open-chat queue has `UNIQUE(send_date, room_name)` as a second guard.

7. **Device blocks reach workers immediately via push.** The push hub listens on Postgres LISTEN/NOTIFY and wakes workers over SSE. When `device_keys` changes, it re-authenticates before waking them. A blocked device gets `revoked` and its connection is closed. On the server side, filling the single `revoked_at` field is enough.

8. **deploy.sh re-runs itself from the version in the commit being deployed.** The 2026-09-28 deploy failure happened because the running script differed from the script in the commit. The image tag is set once as `sha-<commit>`, and if the pull fails it stops instead of falling back to :latest. compose is rendered before the build, so missing variables are caught before a 40-minute build.

Status, judgment, and limits

- In production. The 11 staff PCs and 9 seller-finder workers are counts as registered on 2026-09-16 and 09-18; later counts are not recorded here.

- The most frequent incident was two devices holding the same resource (07-29, 09-22). So device assignment is the default.

- Limits: the Windows collector has no auto-updater and is replaced by reinstalling. The collection-gap alert is off by default.

- If I did it again: I would give the Windows collector a zip hash updater from day one. The 09-16 incident grew because old-version PCs never received the client fix.

What I decided

- If the CI image is missing, the deploy stops instead of substituting :latest. The deployed image always matches a commit.

- During the DB outage, I stood up the same API on a VM before touching the apps. App-side changes came down to connection settings and a few adapters.

- The cutoff-date filter does not drop messages whose timestamp it cannot read. Letting a little extra in is easier to see and fix than data silently disappearing.